

AUDITORÍA DE KPIs

Generado: 05/06/2026 07:09 | Fuente: src/app.jsx | Sin acceso a datos runtime

Este documento audita fórmulas, selectores de estado y campos fuente de cada KPI en CentroOps. **No incluye valores numéricos reales** — la auditoría es sobre código fuente, no datos de ejecución. Para ver IDs exactos de tickets, usar el snippet de consola en Sección 6.

0. Mapa de estados y conjuntos financieros

Flujo lineal de transiciones:

```
recibido → validando | sourcing | cancelado
validando → sourcing | cotizado | cancelado
sourcing → cotizado | cancelado
cotizado → autorizado | cancelado
autorizado → comprado | cancelado
comprado → tránsito | cancelado
tránsito → entregado | cancelado
entregado → facturado | cancelado
facturado → cobrado | cancelado
cobrado → cerrado | cerrado → (terminal) | cancelado → (terminal)
```

Conjuntos financieros (definidos en código, líneas 175–187):

Campo / KPI	Fórmula / Fuente	Observaciones
BACKLOG_SET	{autorizado, comprado, tránsito}	Vendido y comprometido — no entregado aún
CARTERA_SET	{entregado, facturado}	Entregado pero no cobrado
CASH_SET	{cobrado, cerrado}	Dinero efectivamente recibido
OPERADO_SET	{entregado, facturado, cobrado, cerrado}	Trabajo ejecutado — genera revenue
FORECAST_SET	{cotizado, autorizado}	Probable — NO contamina revenue realizado
PIPELINE_SET	{recibido, validando, sourcing, comprado, tránsito}	En proceso, sin entrega aún
CLOSED_SET	{cerrado, cancelado, cobrado}	Estados terminales
earlyPipeline	{recibido, validando, sourcing, cotizado}	Pipeline Comercial — pre-compromiso

■ **SUPERPOSICIÓN:** 'autorizado' pertenece a FORECAST_SET Y a BACKLOG_SET al mismo tiempo. Aparece en Pipeline Comercial (forecast revenue) y en Backlog Operativo (capital comprometido). No hay doble conteo de montos pero sí duplica el conteo de tickets entre secciones.

1. Pipeline Comercial

Selectores (líneas 2162, 2192–2196):

```
earlyPipeline = tickets.filter(t => !t._deleted && status ∈ {recibido,validando,sourcing,cotizado})
forecastTkts = tickets.filter(t => !t._deleted && status ∈ {cotizado,autorizado})
cotizados = forecastTkts.filter(t => t.status === 'cotizado')
autorizados = forecastTkts.filter(t => t.status === 'autorizado')
```

Campo / KPI	Fórmula / Fuente	Observaciones
En pipeline (conteo)	<code>earlyPipeline.length</code>	Tickets pre-compromiso
Cotizados (conteo)	<code>cotizados.length</code>	Pendiente aprobación del cliente
Forecast revenue	$\sum t.snap.precioConIVA$ sobre <code>forecastTks</code>	■ SIN ponderar por <code>t.prob</code>
Forecast utilidad	$\sum utilidadPonderada(t.snap.uNeta, t.prob)$ (uso interno)	Solo usado en contexto interno, no mostrado
Conversión	<code>cobradas.length / active.length × 100</code>	<code>cobradas = {cobrado, cerrado}</code> ; <code>active = !_deleted</code>

■ Forecast revenue no pondera por probabilidad (`t.prob`). Un ticket con `prob='low'` cuenta igual que uno con `prob='high'`. El campo existe pero no se aplica en este KPI.

2. Backlog Operativo

Selector (línea 2253):

```
backlog = tickets.filter(t => !t._deleted && status ∈ {autorizado, comprado, tránsito})
```

Fórmulas exactas (líneas 2254–2259):

Campo / KPI	Fórmula / Fuente	Observaciones
Revenue comprometido	$\sum t.snap.precioConIVA$ (backlog)	Precio final al cliente con IVA incluido
Utilidad comprometida	$\sum t.snap.uNeta$ (backlog)	Utilidad después de gastos y cargas fiscales
Capital comprometido	$\sum t.snap.costoBase \times (1 + t.snap.params.iva/100)$ (backlog)	Costo de producto con IVA. Default iva=16 si falta el campo
Tiempo prom. en backlog	$avg((Date.now() - parseDateMX(t.date)) / 86_400_000)$ (backlog)	■ <code>t.date</code> = fecha CREACIÓN del ticket, no fecha de entrada al estado actual
Ticket más antiguo	$max((Date.now() - parseDateMX(t.date)) / 86_400_000)$ (backlog)	El de mayor antigüedad desde su fecha de creación

■ PROBLEMA CRÍTICO EN AGING: 'Tiempo en backlog' se mide desde `t.date` (creación del ticket), NO desde que el ticket entró al estado autorizado/comprado/tránsito. Un ticket creado hace 30 días y recién autorizado hoy mostrará 30 días de backlog. La app no registra timestamps por estado — solo el timeline de eventos.

3. Cobranza

Selectores:

```
cartera = tickets.filter(t => !t._deleted && status ∈ {entregado, facturado})
vencidos = cartera.filter(t => t.promesaPago && parseDateMX(t.promesaPago) < new Date())
cobrados = tickets.filter(t => !t._deleted && status ∈ {cobrado, cerrado})
```

Campo / KPI	Fórmula / Fuente	Observaciones
Capital en Cartera	$\sum t.snap.precioConIVA$ (cartera)	Revenue pendiente de cobro
Cash cobrado	$\sum t.snap.precioConIVA$ (cobrados)	Dinero efectivamente recibido
Por cobrar	<code>cartera.length</code>	Número de tickets sin cobrar
Pagos vencidos	<code>vencidos.length</code> [<code>promesaPago < hoy</code>]	■ Solo si <code>t.promesaPago</code> registrado
Flujo operativo	<code>carteraMonto - cargaFiscal - cxp</code>	<code>cxp</code> = $\sum snap.costoTotal$ de cartera

Aging <30 / 30-60 / >60d	buckets por antigüedad de t.promesaPago	Solo tickets payType='credit' con promesaPago
--------------------------	---	---

■ Tickets sin t.promesaPago NUNCA se detectan como vencidos aunque lleven meses sin cobrar. La alerta de pagos vencidos está subestimada si existen tickets de crédito sin fecha de promesa.

4. Liquidez Operativa — Capital Inmovilizado

Fórmula principal (línea 2265):

$\text{capitalInmovilizado} = \text{backlogCosto} + \text{carteraMonto}$

Campo / KPI	Fórmula / Fuente	Observaciones
Capital Comprometido	$\sum \text{costoBase} \times (1 + \text{iva} / 100)$ [status: autorizado/comprado/tránsito]	COSTO al proveedor con IVA — no el precio al cliente
Capital en Cartera	$\sum \text{snap.precioConIVA}$ [status: entregado/facturado]	PRECIO al cliente con IVA — dinero pendiente de entrada
Capital Inmovilizado Total	Comprometido (costo) + Cartera (precio)	■ Suma heterogénea: costo + revenue

■ HETEROGENEIDAD: Capital Comprometido usa costos (lo que sale de caja al proveedor) mientras que Capital en Cartera usa revenue (lo que entrará del cliente). La suma es válida como métrica de liquidez total atrapada pero mezcla naturalezas distintas.

5. Auditoría — Primera operación (~2 horas de resolución)

Si este ticket tiene hoy un status en OPERADO_SET (entregado/facturado/cobrado/cerrado), la siguiente tabla indica exactamente en qué métricas participa y en cuáles no:

Métrica	Participa?	Condición exacta
Tiempo prom. en backlog	✗ NO	Solo tickets con status ∈ {autorizado,comprado,tránsito}. Si ya fue entregado/cobrado, está fuera de BACKLOG_SET y queda excluido.
Revenue comprometido	✗ NO	Mismo motivo — backlog = BACKLOG_SET.
Capital Comprometido	✗ NO	Ídem — costoBase solo se suma para tickets en BACKLOG_SET.
Capital en Cartera	✓ SÍ — si status = entregado o facturado	Entra en CARTERA_SET y suma snap.precioConIVA.
Cash cobrado	✓ SÍ — si status = cobrado o cerrado	Suma snap.precioConIVA al cashTotal.
Resolución operativa (%)	✓ SÍ — en ambos conteos	gKpi_total = !_deleted (incluye el ticket). gKpi_concretadas cuenta si status ∈ OPERADO_SET.
Primera respuesta (promedio)	✓ SÍ — si timeline ≥ 2 eventos	Mide (t[1].ts – t[0].ts). Filtra resultados > 168h. Si tiene ≥2 eventos, su tiempo sí promedia.
totalHoras (tiempo operativo)	✓ SÍ — si horasOp > 0	$\sum \text{t.horasOp}$ sobre active (!_deleted). No filtra por status.
Utilidad por hora (uPH)	✓ SÍ — si status ∈ OPERADO_SET y horasOp > 0	uPorHora = utilidadOp / totalHoras. Contribuye al numerador (utilidadOp = OPERADO_SET) y al denominador (totalHoras = active).

6. Snippet de consola — inspección de datos reales

Pega esto en F12 → Consola con la app abierta:

```
const s=JSON.parse(localStorage.getItem('logisolve_state'))||'{}';
const ts=(s.tickets||[]).filter(t=>!t._deleted);
const BACKLOG=new Set(['autorizado','comprado','transito']);
const CARTERA=new Set(['entregado','facturado']);
```

```
const EARLY=new Set(['recibido','validando','sourcing','cotizado']);
// Pipeline Comercial
console.table(ts.filter(t=>EARLY.has(t.status)).map(t=>({id:t.id,status:t.status,titulo:t.titulo})));
// Backlog – tickets + aging desde fecha de creacion
console.table(ts.filter(t=>BACKLOG.has(t.status)).map(t=>{
const d=t.date?.split('/');const dt=d?.length===3?new Date(d[2]+'-'+d[1]+'-'+d[0]):null;
return{id:t.id,status:t.status,titulo:t.titulo,
rev:t.snap?.precioConIVA,util:t.snap?.uNeta,
costo_iva:(t.snap?.costoBase||0)*(1+(t.snap?.params?.iva||16)/100),
dias_desde_creacion:dt?Math.round((Date.now()-dt)/86400000):null;}));
// Capital en Cartera
console.table(ts.filter(t=>CARTERA.has(t.status)).map(t=>
({id:t.id,status:t.status,monto:t.snap?.precioConIVA,promesa:t.promesaPago})));
// Primera operacion (mas antigua por t.date)
const p=ts.slice().sort((a,b)=>{
const f=d=>{const x=d?.split('/');return x?.length===3?
new Date(x[2]+'-'+x[1]+'-'+x[0]):new Date(0);};
return f(a.date)-f(b.date);});
console.log('1ra op:',p[0]?.id,p[0]?.date,p[0]?.status,'horasOp:',p[0]?.horasOp);
console.log('timeline:',p[0]?.timeline);
```

7. Resumen de riesgos y observaciones

ID	Riesgo	Descripción	Sev.
R-01	Aging de backlog engañoso	'Tiempo en backlog' medido desde t.date (creación), no desde entrada al estado actual. Un ticket cotizado hace 2 semanas y autorizado hoy mostrará 14 días.	ALTA
R-02	Superposición 'autorizado'	'autorizado' aparece en Pipeline Comercial (FORECAST_SET) Y en Backlog Operativo (BACKLOG_SET). No hay doble conteo de monto pero sí de tickets visibles.	MEDIA
R-03	Forecast sin ponderación	forecast revenue = Σ precioConIVA sin multiplicar por t.prob. Ticket con prob='low' pesa igual que prob='high'.	MEDIA
R-04	Capital inmovilizado heterogéneo	Comprometido usa costos (proveedor); Cartera usa revenue (cliente). Suma válida como liquidez pero mezcla naturalezas distintas.	MEDIA
R-05	Vencidos ciegos sin promesaPago	Tickets sin t.promesaPago NUNCA se detectan como vencidos. La alerta está subestimada si hay créditos sin fecha registrada.	ALTA
R-06	Sync full-dump	Cada cambio sube TODOS los registros a Supabase. Con >200 tickets generará latencia y consumo excesivo de requests.	BAJA